

IoT Sensör Veri Akışı ve MQTT Mimari Planlaması

Akıllı Tarım Yönetim Sistemi

Mimari Planlama Dokümanı | Hafta 2

5 Mayıs 2026

İçindekiler

1	Giriş	2
2	MQTT Broker Yapılandırması	2
3	Topic Hiyerarşisi ve QoS Seviyeleri	2
4	Python MQTT İstemcisi (paho-mqtt) Entegrasyonu	2
5	PostgreSQL Veritabanı Şema Tasarımı	3
6	Uçtan Uca Veri Akış Diyagramı (DFD)	4
7	Sonuç	4

1 Giriş

Bu doküman, Akıllı Tarım Yönetim Sistemi kapsamında toprak nemi, sıcaklık ve hava durumu IoT sensörlerinden gelen verilerin sisteme entegrasyonu için tasarlanan MQTT tabanlı veri akış mimarisini detaylandırmaktadır. Planlama; broker yapılandırması, topic hiyerarşisi, QoS seviyeleri, güvenlik gereksinimleri, Python MQTT istemcisi entegrasyonu, PostgreSQL veritabanı şeması ve Django backend ile entegrasyon adımlarını kapsamaktadır.

2 MQTT Broker Yapılandırması

Sistemde **Eclipse Mosquitto v2.0+** veya **EMQX** broker'ı kullanılması önerilmektedir. Temel yapılandırma parametreleri aşağıdaki gibidir:

- **Dinleyiciler:** `listener 1883` (TLS'siz iç ağ için), `listener 8883` (TLS zorunlu dış bağlantılar için)
- **Kalıcılık:** `persistence true` ve `persistence_location /mosquitto/data/`
- **Bağlantı Yönetimi:** `max_connections 10000`, `keep_alive 60`, `connection_timeout 30`
- **Dağıtım:** Docker Compose üzerinden `mosquitto:latest` imajı ile stateful volume bağlama

3 Topic Hiyerarşisi ve QoS Seviyeleri

Ölçeklenebilir yapı için hiyerarşi: `farm/{farm_id}/sensor/{sensor_type}/{sensor_id}`

Tablo 1: Örnek Topic ve QoS Dağılımı

Topic	QoS	Retain	Açıklama
<code>farm/01/sensor/soil_moisture/s1</code>	1	Evet	Toprak nemi verisi, kritik teslimat
<code>farm/01/sensor/temperature/t1</code>	1	Evet	Ortam/hava sıcaklığı
<code>farm/01/sensor/weather/wx1</code>	0	Hayır	Yüksek frekanslı hava durumu telemetri
<code>farm/01/cmd/config/#</code>	2	Hayır	Sensör yapılandırma/-komut akışı

4 Python MQTT İstemcisi (paho-mqtt) Entegrasyonu

Sensör verilerini tüketen, ayrıştıran ve PostgreSQL'e yazan arka plan servisi:

```
1 import paho.mqtt.client as mqtt
2 import json, asyncio, asyncpg
3
4 MQTT_HOST = "mqtt.broker.local"
5 MQTT_PORT = 8883
6 DB_POOL = None
7
```

```

8  async def init_db():
9      global DB_POOL
10     # Initialize connection pool for async DB writes
11     DB_POOL = await asyncpg.create_pool("postgresql://user:pass@db:5432/agri_db
12     ")
13
14     async def store_reading(data):
15         async with DB_POOL.acquire() as conn:
16             await conn.execute(
17                 """INSERT INTO sensor_readings (sensor_id, timestamp, value,
18                 quality_flag, raw_payload)
19                 VALUES ($1, $2, $3, $4, $5)""",
20                 data["sensor_id"], data["ts"], data["value"], data.get("quality", 0)
21                 , json.dumps(data)
22             )
23
24     def on_message(client, userdata, msg):
25         try:
26             payload = json.loads(msg.payload.decode())
27             # Parse topic: farm/01/sensor/soil_moisture/s1
28             parts = msg.topic.split("/")
29             payload["sensor_id"] = parts[3]
30             asyncio.run_coroutine_threadsafe(store_reading(payload), loop)
31         except Exception as e:
32             logger.error(f"Data processing error: {e}")
33
34     loop = asyncio.new_event_loop()
35     client = mqtt.Client(callback_api_version=mqtt.CallbackAPIVersion.VERSION2)
36     client.tls_set(ca_certs="ca.crt")
37     client.username_pw_set("farm01_service", "secure_pass")
38     client.on_message = on_message
39     client.connect(MQTT_HOST, MQTT_PORT, 60)
40     client.subscribe("farm/+/sensor/+/+")
41     client.loop_forever()

```

Listing 1: MQTT Tüketici ve Veritabanı Yazma Mantığı (Özet)

5 PostgreSQL Veritabanı Şema Tasarımı

Zaman serisi optimizasyonu ve Django ORM uyumluluğu gözetilen şema:

```

1  CREATE TABLE IF NOT EXISTS farms (
2      id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
3      name VARCHAR(100) NOT NULL,
4      location GEOGRAPHY(POINT, 4326),
5      created_at TIMESTAMPTZ DEFAULT NOW()
6  );
7
8  CREATE TABLE IF NOT EXISTS sensors (
9      id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
10     farm_id UUID REFERENCES farms(id) ON DELETE CASCADE,

```

```

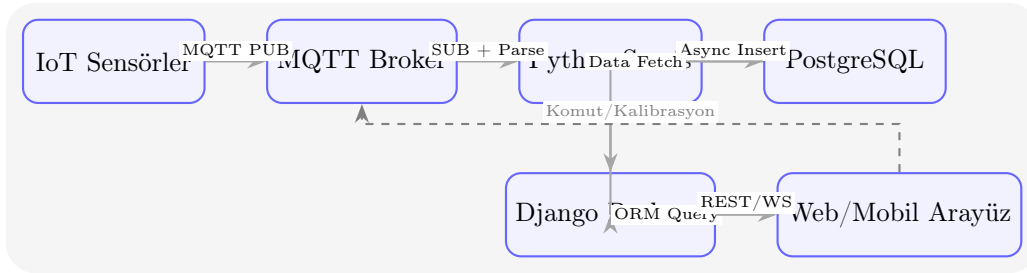
11     sensor_type VARCHAR(50) CHECK (sensor_type IN ('soil_moisture','temperature',
12     , 'weather')),
13     hardware_id VARCHAR(50) UNIQUE NOT NULL,
14     status VARCHAR(20) DEFAULT 'active'
15 );
16
17 CREATE TABLE IF NOT EXISTS sensor_readings (
18     id BIGSERIAL PRIMARY KEY,
19     sensor_id UUID REFERENCES sensors(id),
20     timestamp TIMESTAMPTZ NOT NULL,
21     value FLOAT NOT NULL,
22     quality_flag SMALLINT DEFAULT 0,
23     raw_payload JSONB
24 ) PARTITION BY RANGE (timestamp);
25
26 -- Monthly partition example
27 CREATE TABLE sensor_readings_y2024m01 PARTITION OF sensor_readings
28     FOR VALUES FROM ('2024-01-01') TO ('2024-02-01');
29
30 -- Performance Indexes
31 CREATE INDEX idx_readings_sensor_ts ON sensor_readings(sensor_id, timestamp DESC
32 );
33 CREATE INDEX idx_readings_ts ON sensor_readings(timestamp);

```

Listing 2: Temel Tablo Yapısı ve İndeksler

6 Uçtan Uca Veri Akış Diyagramı (DFD)

Diyagram, sensörden kullanıcı arayüzüne kadar olan veri yolunu ve protokollerini göstermektedir.



Şekil 1: Akıllı Tarım Sistemi Uçtan Uca Veri Akış Diyagramı

7 Sonuç

Hazırlanan mimari planlama; IoT sensörlerinden toplanan ham verilerin güvenli, kayıpsız ve ölçeklenebilir bir şekilde işlenmesini sağlamaktadır. MQTT topic hiyerarşisi ve QoS politikaları ağ trafiğini optimize ederken, TLS+ACL yapısı endüstriyel güvenlik standartlarını karşılar.

Sonraki Adımlar:

1. Mosquitto/EMQX Docker Compose yapılandırmasının hazırlanması
2. Python MQTT servisinin stres testi ve kuyruk mekanizmalarının eklenmesi

3. Django modellerinin migration'ı ve API endpoint'lerinin doğrulanması
4. Pilot çiftlikte 7x24 saatlik yük testi ve latency ölçümü